

RNN

CLASS torch.nn.RNN(*input_size, hidden_size, num_layers=1, nonlinearity='tanh', bias=True, batch_first=False, dropout=0.0, bidirectional=False, device=None, dtype=None*) [SOURCE]

Apply a multi-layer Elman RNN with tanh or ReLU non-linearity to an input sequence. For each element in the input sequence, each layer computes the following function:

$$h_t = \tanh(x_t W_{ih}^T + b_{ih} + h_{t-1} W_{hh}^T + b_{hh})$$

where h_t is the hidden state at time t , x_t is the input at time t , and $h_{(t-1)}$ is the hidden state of the previous layer at time $t-1$ or the initial hidden state at time 0. If `nonlinearity` is `'relu'`, then ReLU is used instead of `tanh`.

```
# Efficient implementation equivalent to the following with bidirectional=False
def forward(x, hx=None):
    if batch_first:
        x = x.transpose(0, 1)
    seq_len, batch_size, _ = x.size()
    if hx is None:
        hx = torch.zeros(num_layers, batch_size, hidden_size)
    h_t_minus_1 = hx
    h_t = hx
    output = []
    for t in range(seq_len):
        for layer in range(num_layers):
            h_t[layer] = torch.tanh(
                x[t] @ weight_ih[layer].T
                + bias_ih[layer]
                + h_t_minus_1[layer] @ weight_hh[layer].T
                + bias_hh[layer]
            )
        output.append(h_t[-1])
        h_t_minus_1 = h_t
    output = torch.stack(output)
    if batch_first:
        output = output.transpose(0, 1)
    return output, h_t
```

Parameters

- input_size** – The number of expected features in the input x
- hidden_size** – The number of features in the hidden state h
- num_layers** – Number of recurrent layers. E.g., setting `num_layers=2` would mean stacking two RNNs together to form a *stacked RNN*, with the second RNN taking in outputs of the first RNN and computing the final results. Default: 1
- nonlinearity** – The non-linearity to use. Can be either `'tanh'` or `'relu'`. Default: `'tanh'`
- bias** – If `False`, then the layer does not use bias weights b_{ih} and b_{hh} . Default: `True`
- batch_first** – If `True`, then the input and output tensors are provided as $(batch, seq, feature)$ instead of $(seq, batch, feature)$. Note that this does not apply to hidden or cell states. See the Inputs/Outputs sections below for details. Default: `False`
- dropout** – If non-zero, introduces a *Dropout* layer on the outputs of each RNN layer except the last layer, with dropout probability equal to `dropout`. Default: 0
- bidirectional** – If `True`, becomes a bidirectional RNN. Default: `False`

Inputs: input, hx

- input:** tensor of shape (L, H_{in}) for unbatched input, (L, N, H_{in}) when `batch_first=False` or (N, L, H_{in}) when `batch_first=True` containing the features of the input sequence. The input can also be a packed variable length sequence. See `torch.nn.utils.rnn.pack_padded_sequence()` or `torch.nn.utils.rnn.pack_sequence()` for details.
- hx:** tensor of shape $(D * num_layers, H_{out})$ for unbatched input or $(D * num_layers, N, H_{out})$ containing the initial hidden state for the input sequence batch. Defaults to zeros if not provided.

where:

N = batch size
 L = sequence length
 D = 2 if `bidirectional=True` otherwise 1
 H_{in} = `input_size`
 H_{out} = `hidden_size`

Outputs: output, h_n

- output:** tensor of shape $(L, D * H_{out})$ for unbatched input, $(L, N, D * H_{out})$ when `batch_first=False` or $(N, L, D * H_{out})$ when `batch_first=True` containing the output features (h_t) from the last layer of the RNN, for each t . If a `torch.nn.utils.rnn.PackedSequence` has been given as the input, the output will also be a packed sequence.

- **h_n**: tensor of shape $(D * \text{num_layers}, H_{out})$ for unbatched input or $(D * \text{num_layers}, N, H_{out})$ containing the final hidden state for each element in the batch.

Variables

- **weight_ih_l[k]** – the learnable input-hidden weights of the k-th layer, of shape $(\text{hidden_size}, \text{input_size})$ for $k = 0$. Otherwise, the shape is $(\text{hidden_size}, \text{num_directions} * \text{hidden_size})$
- **weight_hh_l[k]** – the learnable hidden-hidden weights of the k-th layer, of shape $(\text{hidden_size}, \text{hidden_size})$
- **bias_ih_l[k]** – the learnable input-hidden bias of the k-th layer, of shape (hidden_size)
- **bias_hh_l[k]** – the learnable hidden-hidden bias of the k-th layer, of shape (hidden_size)

• NOTE

All the weights and biases are initialized from $\mathcal{U}(-\sqrt{k}, \sqrt{k})$ where $k = \frac{1}{\text{hidden_size}}$

• NOTE

For bidirectional RNNs, forward and backward are directions 0 and 1 respectively. Example of splitting the output layers when `batch_first=False`: `output.view(seq_len, batch, num_directions, hidden_size)`.

• NOTE

`batch_first` argument is ignored for unbatched inputs.

• WARNING

There are known non-determinism issues for RNN functions on some versions of cuDNN and CUDA. You can enforce deterministic behavior by setting the following environment variables:

On CUDA 10.1, set environment variable `CUDA_LAUNCH_BLOCKING=1`. This may affect performance.

On CUDA 10.2 or later, set environment variable (note the leading colon symbol) `CUBLAS_WORKSPACE_CONFIG=:16:8` or `CUBLAS_WORKSPACE_CONFIG=:4096:2`.

See the [cuDNN 8 Release Notes](#) for more information.

• NOTE

If the following conditions are satisfied: 1) cudnn is enabled, 2) input data is on the GPU 3) input data has dtype `torch.float16` 4) V100 GPU is used, 5) input data is not in `PackedSequence` format persistent algorithm can be selected to improve performance.

Examples:

```
>>> rnn = nn.RNN(10, 20, 2)
>>> input = torch.randn(5, 3, 10)
>>> h0 = torch.randn(2, 3, 20)
>>> output, hn = rnn(input, h0)
```

Docs

Access comprehensive developer documentation for PyTorch
[View Docs >](#)

Tutorials

Get in-depth tutorials for beginners and advanced developers
[View Tutorials >](#)

Resources

Find development resources and get your questions answered
[View Resources >](#)



PyTorch

Get Started

Features

Ecosystem

Blog

Resources

Tutorials

Docs

Discuss

Github Issues

Stay up to date

Facebook

Twitter

YouTube

LinkedIn

PyTorch Podcasts

Spotify

Apple

Google

Amazon

